

- RAG Learning

3.4 Computer Systems

AQA GCSE Computer Science (8525) — Paper 2

3.4.1 Hardware and Software

Hardware is any *physical* component of a computer system — anything you can touch. It includes the parts inside the case (CPU, RAM, hard drive, motherboard, graphics card) and the parts connected to it (keyboard, mouse, monitor, printer, speakers, network router).

Software is the set of *programs* and data that tell the hardware what to do. Software is not a physical object — it is made of instructions, stored as patterns of 0s and 1s on the hardware.

Every working computer is a partnership between the two: hardware provides the physical ability to calculate, store and communicate, but without software it has no instructions and simply sits idle. Software, in turn, is useless without hardware to run it on — a word processor program stored on a disc achieves nothing by itself. Hardware is the stage; software is the performance.

A helpful everyday analogy is a book and its reader. The book (the hardware) is a physical object — paper, binding, ink. The story inside (the software) is the set of ideas that only mean something when someone reads them. Without the book the story has no physical form; without the story the book is just blank paper. Both are needed for the complete experience.

Hardware vs Software — Examples

Hardware (physical)	Software (programs / data)
CPU (processor)	Operating system (e.g. Windows 11, macOS, Linux)
RAM (main memory)	Web browser (e.g. Chrome, Firefox)

Hard disk drive / SSD	Microsoft Word, Excel, PowerPoint
Motherboard	Device drivers (e.g. printer driver)
Monitor, keyboard, mouse	Antivirus software
Graphics card (GPU)	Computer games, music apps, compilers
Network card, router	The compiled binary file of any program

Software Depends on Hardware

Software cannot exist by itself. To run, every piece of software needs:

- A **CPU** to fetch and execute its instructions.
- **Main memory (RAM)** to hold those instructions and data while the program is running.
- **Secondary storage** (HDD/SSD) so the program can be loaded into RAM when needed.
- **Input/output hardware** so the user can interact with the program.

This is why a game designed for a modern PC may refuse to run on an older machine — the CPU is too slow, there is too little RAM, or the graphics card does not support the required features. It is also why demanding software (video editors, games, machine-learning tools) typically list a "minimum" and "recommended" hardware specification on the box.

Exam Tip: If a question asks you to "explain the relationship between hardware and software", aim for three ideas: (1) hardware is physical, software is instructions; (2) software cannot run without hardware to execute it on; (3) hardware is useless without software to give it instructions. That gets you a clean 3 marks.

3.4.2 Boolean Logic

At the most basic level, every circuit inside a computer's processor manipulates **binary** values — 0 and 1. The rules for combining these values are called **Boolean logic**, named after the English

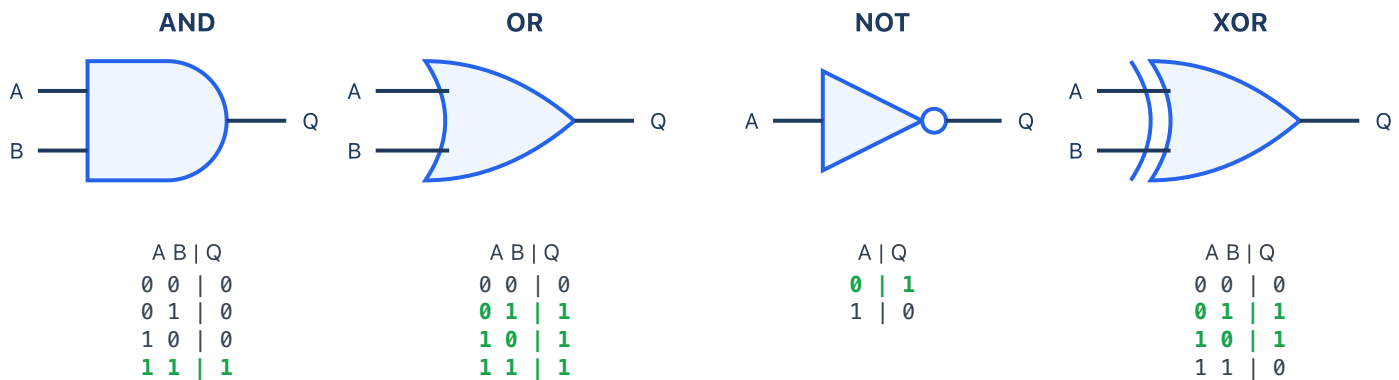
mathematician George Boole. AQA requires you to know four logic operations (also called **gates**): **AND**, **OR**, **NOT** and **XOR**.

A **logic gate** is a small electronic circuit that takes one or more binary inputs and produces a single binary output according to a fixed rule. A **truth table** lists every possible combination of inputs and the matching output. A **logic circuit** is two or more gates wired together to compute a more complex Boolean expression.

The Four Gates

- **AND** — output is 1 only if *both* inputs are 1. Written as $A \text{ AND } B$ or $A \cdot B$.
- **OR** — output is 1 if *at least one* input is 1. Written as $A \text{ OR } B$ or $A + B$.
- **NOT** — takes a single input and inverts it: 0 becomes 1, 1 becomes 0. Written as $\text{NOT } A$ or with a bar: $\bar{A}$.
- **XOR** (exclusive OR) — output is 1 if the inputs are *different*, 0 if they are the same. Written as $A \text{ XOR } B$ or $A \oplus B$.

Standard Logic Gate Symbols and Truth Tables



Quick recall rules (learn these word-for-word):

AND: output is 1 only if BOTH inputs are 1 (think: "&" both must be true).

OR: output is 1 if AT LEAST ONE input is 1 (only 0+0 gives 0).

NOT: inverts a single input — 0 becomes 1, 1 becomes 0.

XOR: output is 1 if the inputs are DIFFERENT (exclusive-OR).

Gate symbol shapes to remember:

AND = D-shape; OR = curved back with pointed output; NOT = triangle + small circle;
XOR = OR with an extra curved line at the input end.

All four gates take 1-bit inputs and produce a single 1-bit output. AQA uses these standard (American) shapes.

Truth Tables for the Four Gates

A truth table has **one row for every possible combination of inputs**. With one input (NOT) there are $2^1 = 2$ rows. With two inputs (AND, OR, XOR) there are $2^2 = 4$ rows. With three inputs there are $2^3 = 8$ rows, and so on.

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

A	NOT A
---	-------

0	1
1	0

Exam Tip: When you write a truth table, always list the input combinations in **binary counting order**: 00, 01, 10, 11 (or 000...111 for three inputs). This guarantees you cover every case and makes the table easy to check. Examiners deduct marks if combinations are missing.

Remembering the Gate Outputs

A useful technique for two-input gates: think about when the output is "unusual" (i.e. 1 rather than 0).

- **AND:** output is 1 only in the *bottom row of the usual table* — the row where both inputs are 1. All other rows give 0.
- **OR:** output is 0 only when *both inputs are 0*. All other rows give 1.
- **XOR:** output is 1 when the inputs *disagree*, 0 when they *agree*. Useful mnemonic: "different = 1".
- **NOT:** just flips the single input.

With these rules memorised, you can fill in any truth table for AND / OR / NOT / XOR without having to re-derive them each time. For longer circuits, build up column by column following the same logic at each stage.

Building Up a Truth Table from a Circuit

For anything more complex than a single gate, you build up the truth table **one gate at a time**. Add a column for the output of each intermediate gate, then combine these columns to get the final output.

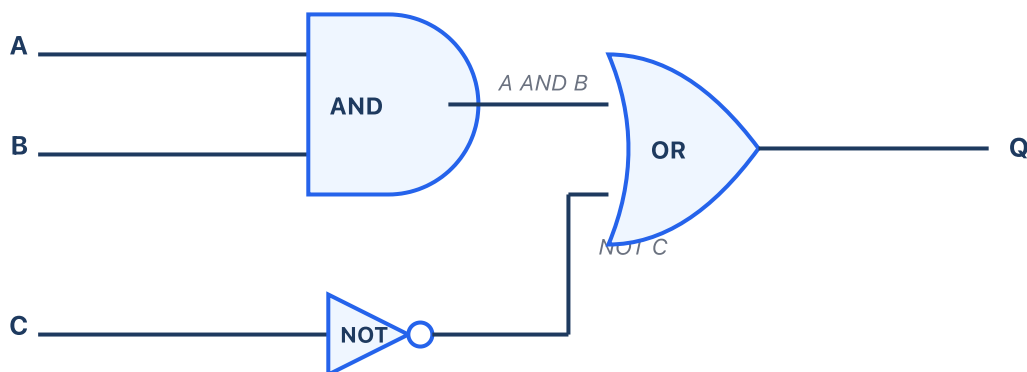
Worked Example — Circuit: $Q = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$

This circuit has three inputs (A, B, C), so the truth table needs $2^3 = 8$ rows. We add intermediate columns for **A AND B** and **NOT C**, then OR them together.

A	B	C	A AND B	NOT C	Q = (A AND B) OR (NOT C)
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Logic Circuit: $Q = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$

Two inputs go through an AND gate; the third is inverted by NOT; both results feed an OR gate.



Reading the circuit:

Signal from A and B is combined by the AND gate. C is inverted by the NOT gate. Both intermediate signals feed an OR gate to produce the output Q.

From a Boolean Expression to a Circuit

To draw a circuit for an expression, work **from the inside out**: handle anything in brackets first, then the outer operators. Each operator becomes one gate. For example:

- $Q = A \text{ AND } (B \text{ OR } C)$ — one OR gate takes B and C; its output and A go into an AND gate.
- $Q = \text{NOT}(A \text{ AND } B)$ — A and B go into an AND gate; that output is fed into a NOT gate.

- $Q = (A \text{ AND } B) \text{ XOR } C$ — A and B feed an AND gate; that output and C feed an XOR gate.

From a Circuit to a Boolean Expression

Working in the opposite direction, label every wire as you trace through the circuit. Output of an AND gate is written $(X \text{ AND } Y)$, output of an OR gate is $(X \text{ OR } Y)$, and so on. Keep combining labels until you reach the final output.

Worked Simplification Example

Consider the expression $Q = (A \text{ AND } B) \text{ OR } (A \text{ AND NOT } B)$. At first glance it has four gates (two AND, one NOT, one OR). But the truth table shows something nice:

A	B	A AND B	A AND NOT B	Q
0	0	0	0	0
0	1	0	0	0
1	0	0	1	1
1	1	1	0	1

The output column is identical to A. So $Q = (A \text{ AND } B) \text{ OR } (A \text{ AND NOT } B) = A$. The four-gate circuit can be replaced with a plain wire. This is the idea behind **circuit simplification** — using algebra or truth tables to find the smallest circuit that produces the same output. Fewer gates means a cheaper, smaller, faster circuit that uses less power, which is why real processor designers spend a lot of effort simplifying logic.

Two-Input Combinations — A Worked Example

Some AQA questions give you a two-input scenario described in English and ask for the Boolean expression. For example: "A lift's doors should open ($D = 1$) only when the lift is stationary ($S = 1$) and either the floor call button (C) or the internal open-door button (B) has been pressed."

Translating this sentence to a Boolean expression gives $D = S \text{ AND } (C \text{ OR } B)$. Two gates are needed: an OR gate combines C and B; its output and S then feed an AND gate.

The truth table for this circuit has three inputs, so $2^3 = 8$ rows:

S	C	B	C OR B	D = S AND (C OR B)
---	---	---	--------	--------------------

0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Notice that every row with $S = 0$ has output 0 (the lift is moving, so doors stay shut regardless). The doors only open in the three rows where $S = 1$ *and* at least one of C or B is 1.

Exam Tip: Reading a circuit in an exam: label each wire as you go — just write "A AND B" on the wire leaving the AND gate. Then follow the arrows. Most marks are lost by skipping this step and trying to read the whole circuit in your head.

3.4.3 Software Classification

Software is traditionally split into two broad families: **system software** (which runs the machine itself) and **application software** (which lets the user do productive work). Every program installed on a computer is one or the other.

System software is software that manages the computer's hardware and provides services for application programs. The user usually does not interact with system software directly — it works quietly in the background. Examples: operating systems, utility software, device drivers, compilers.

Application software is software designed to perform a useful task for the end-user. The

user interacts with it directly. Examples: word processors, web browsers, spreadsheets, games, media players, email clients.

System vs Application Software — Summary

Feature	System software	Application software
Purpose	Manages hardware and provides a platform for other software to run	Performs a specific task useful to the end user
Interacts directly with user?	Usually not — runs in the background	Yes — user interacts with it through a GUI or similar
Essential to run the computer?	Yes — the computer cannot function without it	No — the computer runs without it
Installed by	Usually preinstalled, comes with the OS	Usually installed by the user as needed
Examples	Windows 11, macOS, Linux; antivirus, file compression, defragmentation, device drivers	Microsoft Word, Chrome, Spotify, Minecraft, Photoshop

The Operating System (OS)

The most important piece of system software is the **operating system**. It sits between the hardware and the applications, performing several key jobs:

- **Processor management** — deciding which program gets CPU time next, so many programs can appear to run at once (multitasking).
- **Memory management** — allocating RAM to each running program, preventing them from interfering with each other.
- **File management** — organising data into files and folders on storage, so programs and users can read and save documents.
- **Peripheral / device management** — controlling input and output devices (keyboard, screen, printer, network card) via device drivers.

- **User management / security** — providing user accounts, passwords, permissions and keeping users' files separate.
- **User interface** — providing a GUI (Windows, icons, menus) or command-line for users to interact with.

Utility Software

Utility software is system software that helps to maintain, analyse, configure or protect a computer. Utilities perform specific housekeeping tasks rather than providing a general platform like the OS does.

Common examples required by AQA:

- **Backup utility** — makes copies of files onto another drive or to the cloud, so data can be recovered after loss, theft or damage. Often scheduled to run automatically.
- **Antivirus / anti-malware** — scans files and network traffic for known malicious code, quarantines infected files, and blocks suspicious activity.
- **Disk defragmentation** — on a magnetic hard disk, files become scattered across the disc over time (fragmented). A defragmenter rearranges the data so related blocks are next to each other, speeding up reads. *Not needed on SSDs.*
- **Compression** — creates smaller versions of files or folders (e.g. ZIP, RAR) so they take less storage and transfer faster across networks.
- **Disk cleanup / optimisation** — removes temporary and junk files freeing up disk space.
- **Encryption software** — scrambles data so that only someone with the correct key can read it, protecting confidential information.

Device Drivers — The Hidden System Software

A **device driver** is a specialised piece of system software that lets the operating system talk to a specific piece of hardware — a printer, a graphics card, a webcam. Without the correct driver the hardware either does not work or works only with reduced capability. Drivers are usually installed automatically by the OS, or downloaded from the manufacturer's website. They are invisible to users but essential: plug in a new printer and a quick check of Device Manager will show the driver doing its job behind the scenes.

Exam Tip: A common 4-mark question is: "Give two examples of utility software and state the purpose of each." Answer concisely: name each utility (1 mark) and give a one-sentence purpose (1 mark) — don't give vague answers like "it helps the computer".

3.4.4 Systems Architecture

The overall design of a computer — how the processor, memory and buses are arranged — is called its **architecture**. The dominant design, used by virtually every modern computer from a smartphone to a supercomputer, is the **Von Neumann architecture**, named after the mathematician John von Neumann who described it in 1945.

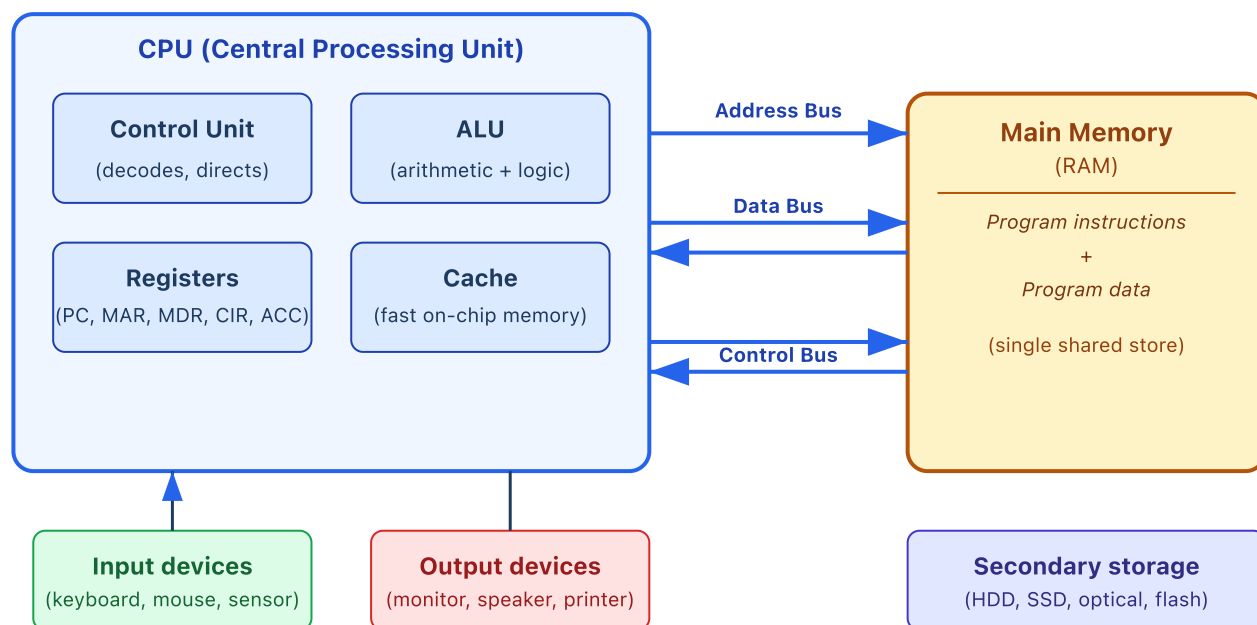
The Von Neumann Architecture — Key Ideas

In the **Von Neumann architecture**, a computer has a **single shared main memory** that holds *both the program's instructions and the data* it operates on. A **central processing unit (CPU)** fetches instructions from memory one at a time, decodes them and executes them. This idea is called the **stored program concept**.

Before Von Neumann, computers had to be physically rewired or have switches reset for each new task — programs were not *data*, they were hardware configurations. Von Neumann's big idea was: treat programs as data, store them in memory alongside the input/output data, and let the CPU fetch whatever instruction is next. A computer could then run any program simply by loading it into memory. Every computer you use today is built on this principle.

Von Neumann Architecture

One shared main memory holds both program instructions and data. The CPU fetches and executes them via three buses.



The three buses:

Address bus: one-way, CPU → memory, carrying the address of the location to read/write.

Data bus: two-way, carrying the actual data or instruction between CPU and memory. **Control bus:** carries control signals like

The Main Components of the CPU

Inside the CPU there are four components you must be able to name and describe for AQA:

Component	Role
Arithmetic Logic Unit (ALU)	Performs all arithmetic (add, subtract, multiply, divide) and logic operations (AND, OR, NOT, XOR, comparisons). All data processing happens here.
Control Unit (CU)	Fetches each instruction from main memory, decodes it, and coordinates the other components by sending out control signals along the control bus. It is the "conductor" of the CPU.
Registers	Tiny, very fast storage locations inside the CPU used to hold the data and instructions currently being worked on. Key examples: Program Counter (PC), Memory Address Register (MAR), Memory Data Register (MDR), Current Instruction Register (CIR), Accumulator (ACC).

Cache	A small amount of very fast memory located on the CPU chip. It stores recently or frequently used instructions and data so the CPU can reuse them without having to go all the way to slower main memory (RAM).
--------------	---

Exam Tip: AQA commonly asks "Describe the role of the Control Unit." A mark-winning answer mentions all three jobs: (1) fetches instructions from memory, (2) decodes each instruction, and (3) controls / coordinates the other components via control signals.

The Three Buses Between CPU and Memory

The CPU communicates with main memory (and other parts of the system) over groups of parallel wires called **buses**. AQA requires you to know three:

Bus	Direction	What it carries
Address bus	One-way: CPU → memory	The <i>memory address</i> the CPU wants to read from or write to (e.g. "location 20").
Data bus	Two-way (CPU ↔ memory)	The actual <i>data or instruction</i> being moved in or out of memory.
Control bus	Two-way	Control signals such as "read", "write", "interrupt", and the clock signal that synchronises everything.

The width of a bus (the number of wires) matters. A 32-bit address bus can address 2^{32} locations — about 4 GB — which is why older 32-bit computers could not use more than 4 GB of RAM. A 64-bit address bus can address 2^{64} locations, far more than is likely ever to be needed. The data bus width similarly limits how many bits can move per transfer.

3.4.5 CPU Performance

Three factors of a CPU are specified by AQA as affecting performance: **clock speed**, **cache size** and **number of cores**. You must be able to describe the effect of each.

Factor	What it is	Effect on performance
Clock speed	The number of clock cycles a CPU performs per second, measured in Hertz (Hz). Modern CPUs run at billions of cycles per second (GHz = 10^9 Hz).	Each instruction takes one or more clock cycles. A higher clock speed means more instructions per second, so programs run faster. However, higher clock speeds generate more heat and use more power.
Cache size	The amount of very-fast memory built into the CPU, measured in MB (typically 1-32 MB).	More cache means more frequently-used instructions and data can be stored close to the CPU. The CPU spends less time waiting for slow RAM, so programs run faster — especially for workloads that reuse the same data.
Number of cores	A core is a complete processing unit on the chip. A dual-core CPU has 2 cores, a quad-core has 4, and so on.	Each core can run its own instructions at the same time as the others (true parallel processing). More cores allow more programs / threads to run simultaneously, improving performance when software is <i>multi-threaded</i> . Software written for a single core cannot use the extra cores, so more cores does not always mean a proportional speedup.

Why "More is Better" Is Not Always True

Each factor has limits:

- **Clock speed** is limited by heat. Doubling the clock speed roughly quadruples the power used and the heat produced. Modern CPUs use techniques like "turbo boost" to briefly increase speed when the cooling system can cope.
- **Cache** is fast but expensive and physically large — designers balance having enough cache to improve performance without making the CPU chip too big or costly.
- **Cores** only speed up software that is written to use them. A game that only runs on one core gains no benefit from having 16 cores.

Comparing Two CPUs — Worked Example

A student is choosing between two laptops:

- **Laptop A:** 4 GHz, 4 cores, 8 MB cache.
- **Laptop B:** 3 GHz, 8 cores, 16 MB cache.

Which is "faster"? It depends on what the student does. If they mostly use one program at a time that is not multi-threaded (many older games, a simple web browser), Laptop A's higher clock speed will tend to feel faster because each core runs more instructions per second. If the student runs many demanding programs at once, or uses software designed for parallel work (video editing, 3D rendering, compiling large projects), Laptop B's extra cores and larger cache give it a significant edge. Clock speed is not the only number that matters — this is why modern CPU specifications always quote **all three** factors.

Exam Tip: For a 4-mark question "describe how [clock speed/cores/cache] affect CPU performance", always give two things: *what changes inside the CPU* and *what the user would notice*. E.g. "A larger cache stores more instructions close to the CPU so the CPU spends less time waiting for RAM, which makes programs run faster overall."

3.4.6 The Fetch-Execute Cycle

Every program is just a long list of instructions in main memory. The CPU runs a program by carrying out the same three-stage cycle over and over, millions of times a second, until the program ends.

The **fetch-execute cycle** (also called the fetch-decode-execute cycle) is the sequence of steps the CPU carries out for every single instruction in a program: **fetch** the next instruction from memory, **decode** it to work out what to do, and **execute** it.

The Key Registers

During the cycle the CPU uses several special-purpose registers:

Register	Full name	Role
----------	-----------	------

PC	Program Counter	Holds the <i>address</i> of the next instruction to fetch. After each fetch it is incremented by 1 so the CPU will fetch the following instruction next time.
MAR	Memory Address Register	Holds the address in main memory that the CPU wants to read from or write to. The address is placed on the address bus.
MDR	Memory Data Register	Holds the data or instruction that has just been read from (or is about to be written to) main memory. Travels on the data bus.
CIR	Current Instruction Register	Holds the current instruction being decoded / executed.
ACC	Accumulator	Temporary store for the result of arithmetic or logic operations performed by the ALU.

The Three Stages — Step by Step

1. Fetch.

- The address in the **PC** is copied into the **MAR**.
- The address is sent via the **address bus** to main memory, and a "read" signal is sent on the control bus.
- Main memory returns the instruction at that address along the **data bus** into the **MDR**.
- The MDR's value is copied into the **CIR**.
- The **PC is incremented** by 1 so it points to the next instruction.

2. Decode.

- The **Control Unit** examines the instruction in the **CIR**.
- It works out what operation is required (e.g. ADD, LOAD, STORE, JUMP) and what operands it uses.

3. Execute.

- The operation is carried out by the appropriate component.
- If the instruction is arithmetic or logic, the **ALU** performs it and the result is stored in the **ACC**.

- If it is a memory access (LOAD/STORE), data is transferred between a register and memory.
- If it is a jump instruction, the **PC** is updated so the next fetch gets the jump target.

The Fetch-Decode-Execute Cycle

Example: executing ADD 40 — instruction at address 20 in memory

STAGE 1 — FETCH

1. PC (20) → MAR. Address bus carries 20 to main memory. Control bus = read.
2. Memory sends instruction at address 20 ("ADD 40") back on the data bus → MDR.
3. MDR → CIR. PC incremented: PC = 21 (ready for next instruction).

PC = 21

MAR = 20

MDR = "ADD 40"

CIR = "ADD 40"

ACC = 10

STAGE 2 — DECODE

The Control Unit examines the instruction in the CIR ("ADD 40").
It determines the operation (ADD) and operand (the value at memory address 40).

PC = 21

CIR = "ADD 40"

ACC = 10

STAGE 3 — EXECUTE

1. CU sends "read address 40" to memory. Memory returns value (say 5) into MDR.
2. ALU adds the value 5 to the current ACC value (10).
3. Result stored back in ACC: ACC = 15.

PC = 21

MAR = 40

MDR = 5

CIR = "ADD 40"

ACC = 15

Instruction complete. Cycle restarts from Fetch, using the new PC (21).

← Fetch-Decode-Execute repeats for every instruction in the program →

A Worked Trace of Four Instructions

Suppose main memory contains the following tiny program starting at address 20, and the ACC starts at 0:

Address	Instruction	
20	LOAD 100	(copy the value at address 100 into ACC)
21	ADD 101	(add the value at address 101 to ACC)
22	STORE 102	(copy ACC into address 102)
23	HALT	

Memory: address 100 = 7
 address 101 = 8

Tracing through:

1. **Cycle 1 (address 20):** Fetch "LOAD 100" into CIR, PC becomes 21. Decode: load from 100. Execute: memory at 100 (= 7) is placed in ACC. ACC = 7.
2. **Cycle 2 (address 21):** Fetch "ADD 101" into CIR, PC becomes 22. Decode: add value at 101 to ACC. Execute: $7 + 8 = 15$. ACC = 15.
3. **Cycle 3 (address 22):** Fetch "STORE 102" into CIR, PC becomes 23. Decode: store ACC at 102. Execute: $\text{memory}[102] = 15$.
4. **Cycle 4 (address 23):** Fetch "HALT", PC becomes 24. Decode and execute: the CPU stops.

Four fetch-decode-execute cycles have run, each one following the same three-stage pattern. Real programs contain millions of instructions, so the cycle repeats millions of times per second — once for every tick (or small number of ticks) of the clock.

Exam Tip: When asked to "describe the fetch-execute cycle", make sure you mention the **PC being incremented** during the fetch stage — this is a common missing mark. A good answer touches each of the three stages and names at least PC, MAR, MDR and CIR.

3.4.7 Memory — RAM and ROM

"Main memory" inside a computer is actually made of two different kinds of chip: **RAM** and **ROM**. Both are fast memory that the CPU can access directly, but they have different purposes.

RAM — Random Access Memory

RAM is the main working memory of the computer. It is **volatile** — its contents are lost when the power is turned off. It is **read/write** — the CPU can both read from it and write to it. It holds the *currently running programs* and the *data they are using*.

When you open a program, the operating system copies it from secondary storage (the hard disk) into RAM so the CPU can execute its instructions quickly. Closing a program frees up that RAM. When the computer is switched off, everything in RAM is lost — which is why unsaved work disappears after a power cut.

Computers with more RAM can hold more programs and data in memory at once. If RAM runs out, the OS has to swap pages of memory to a file on the hard disk ("virtual memory" or the "page file") which is far slower than real RAM, making the system feel sluggish.

ROM — Read-Only Memory

ROM is a memory chip whose contents are fixed when the computer is manufactured (or flashed during an update). It is **non-volatile** — contents survive when the power is off. It is **read-only** — the CPU can read from it but not write to it in normal operation. It typically holds the *boot-up instructions* (the BIOS or firmware).

When a computer is first switched on, the CPU cannot do anything until it finds a program to run. RAM is empty (volatile), and the hard disk cannot read/write without instructions, so the CPU must run the **bootstrap program** stored in ROM. This program tests the hardware (POST — power-on self test), then loads the operating system from the hard disk into RAM.

RAM vs ROM — Comparison

Feature	RAM	ROM
Volatility	Volatile — loses contents when power is off	Non-volatile — keeps contents without power
Read/Write	Read and write	Read-only (in normal use)
Typical size	Gigabytes (e.g. 8 GB, 16 GB, 32 GB)	Small — kilobytes to a few megabytes
Typical contents	Operating system (after boot), running programs, data in use	Boot-up program / BIOS / firmware

Can be upgraded?	Yes — add more RAM sticks	Usually not — fixed at manufacture (firmware may be "flashed")
Speed	Very fast	Fast

Exam Tip: Classic 2-mark question: "State one difference between RAM and ROM." The easiest marks come from volatility: "RAM is volatile (loses its contents when power is off); ROM is non-volatile (keeps its contents without power)." A second mark for read/write vs read-only.

Why Both RAM and ROM Are Needed

Students sometimes ask: "if RAM is bigger and faster, why bother with ROM at all?" The reason is that ROM is **non-volatile** — it remembers what is written to it even when there is no power. When a computer is first switched on, every byte of RAM is random nonsense; the CPU needs an instruction from *somewhere* to do anything at all. ROM is the answer: it is hard-wired with a small program (the BIOS or UEFI firmware) that tells the CPU how to test the hardware and how to load the operating system from the hard disk into RAM. Without ROM, a computer could never boot up; without RAM, it could not run any useful program fast enough once booted.

Virtual Memory — A Brief Mention

When the physical RAM is full, a modern operating system can use a chunk of the hard disk as "pretend RAM" called **virtual memory** or a **page file**. Pages (fixed-size blocks) of memory that have not been used recently are copied out to disk, freeing up real RAM for whatever program needs it. The idea keeps programs running even when they collectively need more memory than is installed, but because disk access is far slower than RAM, a computer that is frequently using virtual memory feels painfully slow — this is called **thrashing**. Adding more RAM typically cures it.

3.4.8 Secondary Storage

Main memory (RAM) is fast but volatile and relatively small. To save files and programs between power-offs, computers need **secondary storage**.

Secondary storage is *non-volatile* storage used to hold programs and data long-term. It is separate from main memory: the CPU cannot execute instructions from it directly — they must first be copied into RAM. Secondary storage is typically **larger, cheaper per byte, and slower** than RAM.

Why Secondary Storage Is Needed

- **Non-volatility** — data survives when the computer is switched off, so documents, photos, programs and the operating system itself are preserved.
- **Capacity** — modern hard drives hold terabytes, far more than is practical in RAM.
- **Cost** — price per byte is a tiny fraction of RAM, making it affordable to store large libraries of files.
- **Portability** — removable media (USB sticks, SD cards, DVDs) let data be moved between computers.

The Three Types of Secondary Storage

1. Magnetic Storage — e.g. Hard Disk Drive (HDD)

An HDD stores data as patterns of magnetised regions on spinning metal platters. A moving read/write head hovers above the surface, and the platter spins at high speed (typically 5,400 or 7,200 RPM). Magnetic tape (used for backups in large data centres) also falls in this category.

- **Advantages:** very high capacity for low cost, long-established technology, good for bulk storage of archives and backups.
- **Disadvantages:** contains moving parts so can fail if dropped or shaken; slower than SSDs; produces some noise and heat; uses more power than flash.

2. Optical Storage — CD, DVD, Blu-ray

Optical media store data as microscopic pits in a reflective layer. A laser reads them — the difference between a pit and a flat (land) changes the reflected light. Capacities range from CD (~700 MB), to DVD (~4.7 GB single-layer), to Blu-ray (~25 GB single-layer).

- **Advantages:** cheap per disc, portable, widely compatible, good for distributing software and films.

- **Disadvantages:** low capacity compared with HDDs and SSDs; slow read/write; easy to scratch; read speed much slower than flash or HDD; many modern laptops no longer have optical drives.

3. Solid-State Storage — SSD, USB Flash Drive, SD Card

Solid-state storage uses **flash memory** — silicon chips with no moving parts. Each cell stores bits by trapping electrons in a floating gate. SSDs fit into the same slots as HDDs but are much faster; USB sticks and SD cards use the same technology in more portable form factors.

- **Advantages:** very fast (no seek time), silent, durable (no moving parts so survive drops), low power use, small and light.
- **Disadvantages:** more expensive per GB than HDD; flash cells have a limited number of write cycles (thousands to around 100,000, but finite); capacity currently lower at the top end than the largest HDDs.

Comparison Table — HDD vs SSD vs Optical vs Flash

Feature	HDD (magnetic)	SSD (solid state)	Optical (DVD/Blu-ray)	Flash (USB/SD)
Speed	Moderate	Very fast	Slow	Fast (slower than an internal SSD)
Capacity	Very high (TBs)	High (hundreds of GB to multi-TB)	Low (~700 MB CD / 4.7 GB DVD / 25 GB Blu-ray)	Low-to-medium (usually GBs)
Cost per GB	Very low	Higher than HDD	Very low (per disc)	Moderate
Durability	Fragile (moving parts)	Very durable (no moving parts)	Surface easily scratched	Very durable

Portability	Heavy (internal); external HDD portable but bulky	Very portable internally; external SSDs are small	Highly portable (a disc)	Extremely portable (pocket-size)
Power use	Higher (motor spinning platter)	Low	Moderate (when drive is spinning)	Very low
Typical use	Bulk storage, backups, desktops, servers	OS drive, laptops, fast gaming storage	Distributing films and software; archival	Moving files between devices; camera storage

Choosing the Right Storage — Worked Scenarios

- **A laptop for a student to take to lessons:** SSD. Durable (survives being dropped in a bag), silent, fast boot, low battery drain.
- **A server for a small business to store six years of customer files:** HDD (or a RAID array of HDDs). Cheap per GB, huge capacity, speed is not critical for archive access.
- **Distributing a film to customers:** Blu-ray disc. Cheap to mass-produce, reliable, widely compatible with TV players.
- **Moving a 2 GB report from home to school:** USB flash drive. Pocket-sized, universally supported by USB-A/C ports.
- **A professional photographer's camera:** SD card (flash). Fast writes for high-resolution images, durable, removable.

Exam Tip: When a question asks you to recommend a storage type, it is **never enough** just to name one — you must justify it using at least two relevant features from the list above (speed, capacity, cost, durability, portability, power). Link each feature back to the scenario ("durable so it survives being thrown in a bag").

Case Study — Why Laptops Moved from HDDs to SSDs

Ten years ago almost every laptop used a 2.5-inch hard disk drive. Today nearly every new laptop ships with an SSD. The reason is not a single feature but the combination of many small advantages adding up:

- An SSD has **no moving parts**, so it can survive being knocked about in a bag — an HDD with a spinning platter is easily damaged by a single drop.
- An SSD uses **less power**, noticeably extending battery life because there is no motor to spin.
- An SSD is **silent** and produces **less heat**, so the laptop runs quieter and the fan spins less often.
- Boot times and program launches are **dramatically faster**, because an SSD has essentially zero seek time — it does not need to move a head and wait for a platter to rotate to the right place.
- SSDs are **smaller and lighter**, enabling thinner and lighter laptop designs.

The main downside is **cost per GB**, so budget laptops may still offer a smaller SSD (e.g. 256 GB) where a similarly-priced HDD would have been a terabyte. Large files (videos, photo libraries) are often off-loaded to an external HDD or the cloud.

3.4.9 Embedded Systems

An **embedded system** is a computer system that is built into a larger device to perform a specific, dedicated function. Unlike a general-purpose computer, an embedded system is not designed to run arbitrary software — its hardware and software are specialised to do one job well, reliably and efficiently.

Embedded systems are everywhere and vastly outnumber "normal" computers. Most are invisible to the people using them: the "computer" inside a washing machine, a microwave, a car's engine management unit, a smart thermostat, a pacemaker, a pair of headphones, or an industrial robot. Each of these devices has a small processor, some ROM containing its dedicated program (the *firmware*), a little RAM, and specific input/output hardware for its task (sensors, buttons, motors, displays).

Common Examples

- **Washing machine.** Sensors detect water level and temperature; the embedded controller runs the chosen wash program, timing each stage and actuating valves, heaters and the drum motor.
- **Car engine control unit (ECU).** Reads sensors for air flow, oxygen, engine temperature and throttle position, then computes how much fuel to inject and when to fire the spark plugs, in real time, hundreds of times per second.
- **Smart thermostat.** Takes inputs from temperature sensors and a user's phone app, and controls the heating/cooling output to keep the room at the set temperature, often learning patterns over time.
- **Microwave oven.** Reads the keypad, shows a countdown on the display, controls the magnetron and turntable motor, and sounds an alarm when done.
- **Medical equipment.** Heart-rate monitors, insulin pumps, pacemakers — each runs dedicated firmware with strong reliability and safety requirements.
- **Consumer electronics.** Smart TVs, set-top boxes, digital cameras, smart speakers, fitness trackers.
- **Traffic light controller.** Runs a fixed cycle of signal patterns, reacts to pedestrian buttons and inductive loops in the road surface, and keeps strict timing to coordinate with neighbouring junctions.

Why Embedded Systems Use Specialised Hardware and Software

Because an embedded system only has to do one job, its hardware and software can be heavily tailored to that job:

- **Low cost** — no need for an expensive general-purpose CPU; a simple microcontroller is enough. A washing machine does not need a 16-core desktop processor.
- **Small size and low power** — important when the system must fit into a fridge or run on a small battery for years.
- **Reliability** — fewer features and a dedicated program mean fewer things can go wrong; the firmware is tested specifically for that device.
- **Real-time response** — an embedded ECU must react to sensors within milliseconds, so the software is stripped of anything that might add delay (no multitasking OS, etc.).

- **Fixed program** — the software is usually stored in ROM/flash and changes only via a manufacturer's firmware update. Users cannot install arbitrary software, reducing security and stability risks.

Embedded System vs General-Purpose Computer

Feature	Embedded system	General-purpose computer
Purpose	Performs one specific task	Can run any software the user chooses
Hardware	Specialised microcontroller + task-specific sensors/actuators	Powerful general CPU, large RAM, many peripheral ports
Software	Fixed firmware in ROM / flash, rarely updated	OS plus user-installable applications, updated frequently
User interface	Minimal — a few buttons, a small screen, or none at all	Rich — keyboard, mouse, high-res display, GUI
Cost	Cheap to mass-produce	Relatively expensive
Power use	Very low; may run on batteries for years	Higher; usually mains-powered
Examples	Washing machine, car ECU, thermostat, pacemaker	Desktop PC, laptop, smartphone, server

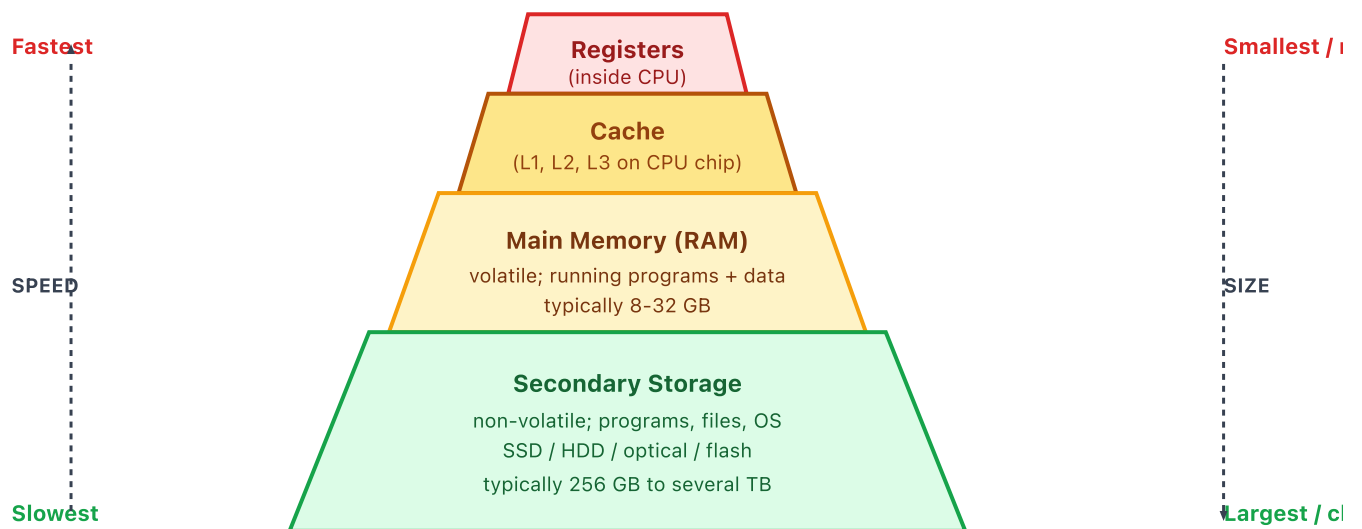
Exam Tip: A common exam question: "Give an example of an embedded system and explain why a specialised design is used rather than a general-purpose computer." Pick one concrete example (e.g. "a washing machine"), then give two reasons linked to the example (e.g. "it only needs to run one dedicated wash-cycle program, so a cheap microcontroller is sufficient; it must respond to sensors in real time, which a general OS might fail to guarantee").

3.4 — Memory Hierarchy Big Picture

Finally, here is how all the different types of memory and storage fit together. Closer to the CPU means faster but smaller and more expensive per byte; further away means slower but larger and cheaper per byte.

The Memory Hierarchy

Closer to the CPU = faster, smaller, more expensive per byte. Further away = slower, larger, cheaper per byte.



Indicative access times:

Registers: ~1 CPU cycle • Cache: ~5-50 cycles • RAM: ~100 cycles • SSD: ~100,000 cycles • HDD: ~10,000,000 cycles.

A cache hit is roughly a million times faster than a hard-disk read, which is why caching matters so much for performance.

Understanding this hierarchy explains many of the topics you have met: why more RAM makes a computer feel faster (less disk swapping), why SSDs feel vastly snappier than HDDs (fewer cycles per access), why cache size affects CPU performance (reduces wasted cycles waiting for RAM), and why registers are used for the most active data during the fetch-execute cycle.

Exam Tip: A perennial exam favourite: "State one similarity and one difference between main memory (RAM) and secondary storage." Similarity: both can be read from and written to (accept: both store data and/or programs). Differences: RAM is volatile/secondary is non-volatile; RAM is faster/much smaller; CPU executes directly from RAM but not directly from secondary storage. Any one of those differences is worth 1 mark.

